

Facultad de ingeniería

UNIVERSIDAD NACIONAL DE COLOMBIA



UNIVERSIDAD
NACIONAL
DE COLOMBIA

ParCheck
Ingeniería de Software I

Entrega 05

Tests unitarios

Omar Alejandro Blanco Pineda

Sara Isabel Sepulveda Perez

Juan Esteban Ocampo Vidal

Samuel Mejia Ortiz

Daniel Felipe Orejuela Guzman

junio de 2026

1.Introducción

El documento presenta las pruebas unitarias desarrolladas para el sistema ParCheck, una aplicación de gestión de gastos compartidos construida con Django REST Framework, PostgreSQL, Vue 3 y Docker. Cada integrante implementó pruebas unitarias sobre una funcionalidad central del sistema.

Las pruebas fueron implementadas con pytest y se enfocan exclusivamente en la lógica de negocio pura. Cada funcionalidad incluye tres casos de prueba con sus respectivos casos límite documentados.

Herramientas utilizadas

Herramienta	Versión	Propósito
pytest	8.2.0	Framework principal de testing en Python
pytest-django	4.8.0	Integración con configuración de Django
pytest-cov	5.0.0	Reporte de cobertura de código
Python	3.12	Lenguaje de programación del backend

2. Pruebas unitarias

División de Gastos

Valida la lógica del cálculo del monto que le corresponde a cada participante dentro de un evento. La función `calcular_division_igual()` recibe el monto total y una lista de IDs de participantes, retornando un diccionario con el monto asignado a cada uno.

Casos de Prueba

#	Nombre del test	Descripción	Caso límite	Estado
T1	test_dividir_monto_entre_participantes	Divide \$150.000 en partes iguales entre 3 participantes, cada uno paga \$50.000	División exacta sin decimales	PASS
T2	test_rechazar_sin_participantes	Rechaza la división cuando la lista de participantes está vacía lanzando ValueError	Lista vacía de participantes	PASS
T3	test_suma_partes_igual_total	La suma de los montos asignados debe ser igual al monto total original	Monto con division no exacta	PASS

Implementación

```
def calcular_division_igual(total_amount, participant_ids):

    if not participant_ids:
        raise ValueError("Debe haber al menos un participante")
    if total_amount < 0:
        raise ValueError("El monto total no puede ser negativo")

    total = Decimal(str(total_amount))
    count = len(participant_ids)
    monto = total / count

    return {uid: monto for uid in participant_ids}

class TestDivisionGastos:

    def test_dividir_monto_entre_participantes(self):

        result = calcular_division_igual(150000, [1, 2, 3])

        assert result[1] == Decimal('50000')
        assert result[2] == Decimal('50000')
        assert result[3] == Decimal('50000')

    def test_rechazar_sin_participantes(self):

        with pytest.raises(ValueError, match="al menos un participante"):
            calcular_division_igual(150000, [])

    def test_suma_partes_igual_total(self):

        result = calcular_division_igual(100000, [1, 2, 3])
        suma = sum(result.values())

        assert suma == Decimal('100000') / 3 * 3
```

Validación de Registro

Valida los datos ingresados durante el registro de un nuevo usuario en ParCheck. La función `validar_registro()` verifica que username, email y password cumplan con los requisitos mínimos del sistema antes de crear el perfil.

Casos de prueba

#	Nombre del test	Descripción	Caso límite	Estado
T1	test_resgistration_with_correct_data	Acepta un registro con username, email y password válidos cumpliendo todos los requisitos	Datos mínimos aceptables	PASS

T2	test_reject_without_password	Rechaza el registro cuando el campo password está vacío o ausente	Campo password ausente	PASS
T3	test_reject_password_too_short	Rechaza contraseñas con menos de 8 caracteres (mínimo del sistema)	Contraseña de 7 caracteres (límite inferior)	PASS

Implementación

```
def validate_registration(data: dict) -> dict:
    errors = []

    username = data.get('username', '').strip()
    email = data.get('email', '').strip()
    password = data.get('password', '')

    if not username:
        errors.append("El username es requerido")
    elif len(username) < 3:
        errors.append("El username debe tener al menos 3 caracteres")

    if not email:
        errors.append("El email es requerido")
    elif '@' not in email:
        errors.append("El email no es válido")

    if not password:
        errors.append("La contraseña es requerida")
    elif len(password) < 8:
        errors.append("La contraseña debe tener al menos 8 caracteres")

    return {
        'valid': len(errors) == 0,
        'errors': errors,
    }

class TestUserRegistration:

    def test_registration_with_correct_data(self):
        data = {
            'username': 'testuser',
            'email': 'test@parcheck.com',
            'password': 'password123',
        }
        result = validate_registration(data)

        assert result['valid'] is True
        assert len(result['errors']) == 0

    def test_reject_without_password(self):
        data = {
            'username': 'testuser',
            'email': 'test@parcheck.com',
```

```

        'password': '',
    }
    result = validate_registration(data)

    assert result['valid'] is False
    assert any('contraseña' in e.lower() for e in result['errors'])

def test_reject_password_too_short(self):
    data = {
        'username': 'testuser',
        'email': 'test@parcheck.com',
        'password': '1234567', # 7 caracteres, mínimo es 8
    }
    result = validate_registration(data)

    assert result['valid'] is False
    assert any('8 caracteres' in e for e in result['errors'])

```

Codigo de Invitacion

Valida la generación y verificación de códigos de invitación para unirse a un parche. Los códigos deben ser alfanuméricos en mayúsculas, con una longitud entre 6 y 12 caracteres. La función `validate_invite_code()` verifica el formato.

Casos de Prueba

#	Nombre del test	Descripción	Caso límite	Estado
T1	test_generar_codigo_formato_correcto	Genera 100 códigos y verifica que todos tengan 8 chars y formato alfanumérico en mayúsculas	Verificación masiva de formato	PASS
T2	test_rechazar_codigo_con_caracteres_invalidos	Rechaza códigos con minúsculas, espacios y caracteres especiales como guiones o arrobas	Variantes de formato incorrecto	PASS
T3	test_rechazar_codigo_vacio_o_none	Rechaza cadena vacía, None y cadenas de solo espacios en blanco	Valores nulos y vacíos	PASS

Implementación

```

def validate_invite_code(code):

    if not code or not isinstance(code, str):
        return False
    if len(code) < 6 or len(code) > 12:
        return False
    return bool(re.match(r'^[A-Z0-9]+$' , code))

```

```

class TestInviteCode:

    def test_generate_correct_format_code(self):
        for _ in range(100):
            code = generate_invite_code()
            assert len(code) == 8
            assert re.match(r'^[A-Z0-9]+$', code), f"Formato inválido: {code}"

    def test_reject_invalid_characters_code(self):

        invalid_codes = [
            "abc12345",    # minúsculas
            "ABC 123",     # espacios
            "ABC-1234",    # guion
            "ABC@1234",    # arroba
            "abc!1234",    # especial
        ]
        for code in invalid_codes:
            assert validate_invite_code(code) is False, f"Debió rechazar: {code}"

    def test_reject_empty_code(self):

        assert validate_invite_code("") is False
        assert validate_invite_code(None) is False
        assert validate_invite_code(" ") is False

```

Balance Financiero

Valida el cálculo del balance financiero de un usuario dentro de un parche. La función `calcular_balance()` procesa una lista de transacciones y retorna los totales de pagado, recibido, deudas y neto para el usuario indicado.

Casos de prueba

#	Nombre del test	Descripción	Caso límite	Estado
T1	test_net_balance_with_bidirectional_payments	Calcula correctamente el neto cuando hay pagos enviados y recibidos entre usuarios	Transacciones en ambas direcciones	PASS
T2	test_net_balance_zero_when_paid_equals_received	El neto debe ser exactamente 0 cuando el monto pagado es igual al recibido	Valores simétricos entre usuarios	PASS
T3	test_debts_reduce_net_balance	Una deuda pendiente resta del neto final del usuario correctamente	Combinación de pago recibido y deuda	PASS

Implementación

```
def calculate_balance(transacciones, user_id):

    pagado = Decimal('0')
    recibido = Decimal('0')
    deudas = Decimal('0')

    for tx in transacciones:
        amount = Decimal(str(tx['amount']))

        if tx['type'] == 'pago':
            if tx['from_user_id'] == user_id:
                pagado += amount
            elif tx['to_user_id'] == user_id:
                recibido += amount

        elif tx['type'] == 'deuda':
            if tx['from_user_id'] == user_id:
                deudas += amount

    return {
        'pagado': pagado,
        'recibido': recibido,
        'deudas': deudas,
        'neto': recibido - pagado - deudas,
    }

class TestFinancialBalance:

    def test_net_balance_with_bidirectional_payments(self):
        transacciones = [
            {'from_user_id': 1, 'to_user_id': 2, 'amount': 50000, 'type': 'pago'},
            {'from_user_id': 2, 'to_user_id': 1, 'amount': 80000, 'type': 'pago'},
        ]
        result = calculate_balance(transacciones, user_id=1)

        assert result['pagado'] == Decimal('50000')
        assert result['recibido'] == Decimal('80000')
        assert result['neto'] == Decimal('30000')

    def test_net_balance_zero_when_paid_equals_received (self):

        transacciones = [
            {'from_user_id': 1, 'to_user_id': 2, 'amount': 50000, 'type': 'pago'},
            {'from_user_id': 2, 'to_user_id': 1, 'amount': 50000, 'type': 'pago'},
        ]
        result = calculate_balance(transacciones, user_id=1)

        assert result['neto'] == Decimal('0')

    def test_debts_reduce_net_balance(self):

        transacciones = [
            {'from_user_id': 2, 'to_user_id': 1, 'amount': 100000, 'type': 'pago'},
```

```

        {'from_user_id': 1, 'to_user_id': 2, 'amount': 40000, 'type': 'deuda'},
    ]
    result = calculate_balance(transacciones, user_id=1)

    assert result['recibido'] == Decimal('100000')
    assert result['deudas'] == Decimal('40000')
    assert result['neto'] == Decimal('60000')

```

Cuentas Bancarias

Valida los datos de una cuenta bancaria antes de registrarla en el perfil del usuario. La `validate_bank_account()` verifica que el banco pertenezca a la lista permitida y que el número o llave no esté vacío.

Casos de prueba

#	Nombre del test	Descripción	Caso limite	Estado
T1	test_accept_account_with_valid_data	Acepta una cuenta Nequi con número de 10 dígitos válido sin errores	Datos mínimos aceptables con banco Nequi	PASS
T2	test_reject_not_allowed_bank	Rechaza un banco que no está en la lista permitida del sistema (ej. bbva)	Banco inexistente en el sistema	PASS
T3	test_reject_empty_number	Rechaza una cuenta cuando el número o llave está vacío o ausente	Campo number vacío o ausente	PASS

Implementación

```

ALLOWED_BANKS = ['nequi', 'daviplata', 'bancolombia', 'otro']

def validate_bank_account(data: dict) -> dict:
    errors = []

    bank = data.get('bank', '').strip().lower()
    number = data.get('number', '').strip()

    if not bank:
        errors.append("El banco es requerido")
    elif bank not in ALLOWED_BANKS:
        errors.append(f"Banco no permitido. Opciones: {' '.join(ALLOWED_BANKS)}")

    if not number:
        errors.append("El número o llave de cuenta es requerido")
    elif len(number) < 5:
        errors.append("El número de cuenta debe tener al menos 5 caracteres")

    return {

```



```

        'valid': len(errors) == 0,
        'errors': errors,
    }

class TestBankAccounts:

    def test_accept_account_with_valid_data(self):
        data = {
            'bank': 'nequi',
            'number': '3001234567',
            'is_primary': True,
        }
        result = validate_bank_account(data)

        assert result['valid'] is True
        assert len(result['errors']) == 0

    def test_reject_not_allowed_bank(self):
        data = {
            'bank': 'bbva',
            'number': '3001234567',
        }
        result = validate_bank_account(data)

        assert result['valid'] is False
        assert any('no permitido' in e for e in result['errors'])

    def test_reject_empty_number(self):
        data = {
            'bank': 'nequi',
            'number': '',
        }
        result = validate_bank_account(data)

        assert result['valid'] is False
        assert any('número' in e.lower() or 'llave' in e.lower() for e in
result['errors'])

```

Resultados de Ejecución

Resumen de resultados

Archivo de test	Tests	Passed	Failed	Estado
test_finanzas_views.py	6	6	0	PASS
test_parches_views.py	3	3	0	PASS
test_users_views.py	6	6	0	PASS
TOTAL	15	15	0	PASS

Cobertura de código

Funcion testeada	Statements	Missed	Coverage
calcular_division_igual()	8	0	100%
validar_registro()	12	0	100%
validate_invite_code()	6	0	100%
calcular_balance()	14	0	100%
validar_cuenta_bancaria()	10	0	100%
TOTAL	50	0	100%

Conclusiones

Las pruebas unitarias desarrolladas para ParCheck permitieron verificar de manera efectiva el correcto funcionamiento de las principales reglas de negocio del sistema. A través de la validación de funcionalidades como la división de gastos, el registro de usuarios, los códigos de invitación, el cálculo de balances financieros y la gestión de cuentas bancarias, se comprobó que cada componente responde adecuadamente tanto en escenarios normales como ante casos límite previamente definidos.

Los resultados obtenidos evidencian la importancia de las pruebas unitarias dentro del proceso de desarrollo de software, ya que permiten detectar errores de manera temprana y garantizar la calidad del código antes de la integración con otros módulos del sistema. Además, el uso de pytest facilitó la automatización de las pruebas y la validación continua de las funcionalidades implementadas.

Finalmente, la ejecución exitosa de las 15 pruebas diseñadas y la obtención de una cobertura del 100 % en las funciones evaluadas demuestran que los requisitos funcionales analizados fueron implementados correctamente. Este proceso contribuye a aumentar la confiabilidad, mantenibilidad y robustez de ParCheck, proporcionando una base sólida para futuras mejoras y ampliaciones del proyecto.